Arguments, Premises, Conclusions and Validity

A 1.5–2 hour lecture covering formal arguments, inference rules, validity and CS reasoning.

• Recommended teaching duration: approximately 1.5–2 hours including board work, worked examples, Python demonstrations and student discussion.

• Teaching approach: concept → notation → worked example → CS interpretation → Python verification → student practice.

Lecture Roadmap

• Concept introduction and terminology

• Detailed definitions and notation

• Worked mathematical examples

• CS case studies with complete solutions

• Python implementation and verification

• Extended question bank

• 2–3 unsolved student practice questions at the end

1. Argument and Logic

An argument contains premises and a conclusion. An argument is valid when there is no possible case in which all premises are true while the conclusion is false.

Validity concerns structure, not whether the premises happen to be true in a particular real-world situation.

2. Premise and Conclusion

Words such as because, since, and given that often introduce premises; therefore, hence, and so often introduce conclusions. In formal work, label them clearly.

3. Modus Ponens

Form: $p \to q$, p, therefore q. Example: If a user is authenticated, then the session has a valid token. The user is authenticated. Therefore the session has a valid token.

4. Modus Tollens

Form: $p \to q$, $\neg q$, therefore $\neg p$. Example: If a build succeeds, an executable is produced. No executable was produced. Therefore the build did not succeed, under the stated assumptions.

5. Other Useful Forms

Hypothetical syllogism: $p \to q$, $q \to r$, therefore $p \to r$. Disjunctive syllogism: $p \vee q$, $\neg p$, therefore q.

6. Invalid Arguments

Affirming the consequent and denying the antecedent are common mistakes. p→q, q therefore p is invalid because another cause may produce q. p→q, ¬p therefore ¬q is also invalid.

7. CS Case Study — Debugging

Claim: If the API server is running, health endpoint returns 200. Health endpoint returns 200. Therefore server is running.

Solution: this is p→q, q, therefore p: affirming the consequent. It is invalid because a proxy, cache or mock endpoint could return 200.

8. CS Case Study — Security

Policy: If a login comes from an untrusted device, MFA is required. A login did not require MFA. Therefore it was not from an untrusted device.

Solution: u→m, ¬m, therefore ¬u. This is Modus Tollens and is valid under the policy.

9. Validity by Counterexample

To disprove validity, find one assignment where all premises are true and conclusion false. This is computationally useful for small Boolean arguments.

10. Python Verification

A Python script can enumerate truth assignments, evaluate premises, and search for a counterexample.

Python Laboratory

Python Activity 1

```
from itertools import product

# Test Modus Ponens
for p, q in product([False, True], repeat=2):
    premise1 = (not p) or q   # p -> q
    premise2 = p
    conclusion = q
    if premise1 and premise2:
        print("Premises true:", p, q, "Conclusion:", conclusion)
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Python Activity 2

```
# Search for a counterexample to p->q, q therefore p
for p, q in product([False, True], repeat=2):
    premises = ((not p) or q) and q
    conclusion = p
    if premises and not conclusion:
        print("Counterexample:", p, q)
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Extended Question Bank — Instructor-Led Practice

The following questions are designed to provide enough board/classroom practice for a 1.5–2 hour session. They cover the same topic areas as the relevant VU MTH202 sequence, but are written as fresh exercises rather than reproducing the handout wording.

1. Identify premises and conclusion in a software requirement argument.

2. Determine whether p→q, p, therefore q is valid.

3. Determine whether p→q, ¬q, therefore ¬p is valid.

4. Explain why p→q, q, therefore p is invalid.

5. Create a CS example of denying the antecedent.

6. Use a truth table to test an argument.

7. Write Python code to find a counterexample to an invalid argument.

8. Explain the difference between validity and truth.

9. Analyze an argument about database availability.

10. Analyze an argument about user authentication.

11. Give an example where a health-check response does not prove the underlying server is running.

12. Convert a natural-language security policy into premises and a conclusion.

End-of-Lecture Student Practice — No Solutions

1. Analyze the validity of an argument from a software requirement.

2. Find a counterexample to an invalid implication argument.

3. Write a Python checker for a two-premise argument.

Quick Recap

Arguments formalize reasoning; validity depends on logical structure.